Cryptography and network security for data privacy

EXP 1

```python
def caesar_encrypt(text, key):
    result = ""
    for ch in text:
        if ch.isalpha():
            result += chr((ord(ch.upper()) - 65 + key) % 26 + 65)
        else:
            result += ch
    return result

text = input("Enter text: ")
key = int(input("Enter key: "))

print("Cipher Text:", caesar_encrypt(text, key))
```

```
Enter text: HELLO
Enter key: 3
Cipher Text: KHOOR

=== Code Execution Successful ===
```

EXP2-Monoalphabetic Substitution Cipher

```python
plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = "QWERTYUIOPASDFGHJKLZXCVBNM"

text = input("Enter text: ").upper()
result = ""

for ch in text:
    if ch in plain:
        result += cipher[plain.index(ch)]
    else:
        result += ch

print("Cipher Text:", result)
```

```
Enter text: HELLO
Cipher Text: ITSSG

=== Code Execution Successful ===
```

EX3-Polyalphabetic Cipher

main.py

Share Run

Output

```
def vigenere(text, key):
    result = ""
    key = key.upper()
    j = 0

    for ch in text.upper():
        if ch.isalpha():
            shift = ord(key[j % len(key)]) - 65
            result += chr((ord(ch) - 65 + shift) % 26 + 65)
            j += 1
        else:
            result += ch
    return result

text = input("Enter text: ")
key = input("Enter key: ")

print("Cipher Text:", vigenere(text, key))
```

```
Enter text: HELLO
Enter key: KEY
Cipher Text: RIJVS

=== Code Execution Successful ===
```

EX4-Playfair Cipher

main.py

Share Run

Output

```
def playfair(text):
    text = text.upper().replace("J", "I")
    if len(text) % 2 != 0:
        text += "X"

    print("Pairs:")
    for i in range(0, len(text), 2):
        print(text[i:i+2])

msg = input("Enter message: ")
playfair(msg)
```

```
Enter message: HELLO
Pairs:
HE
LL
OX

=== Code Execution Successful ===
```

EX5-Affine Cipher

main.py

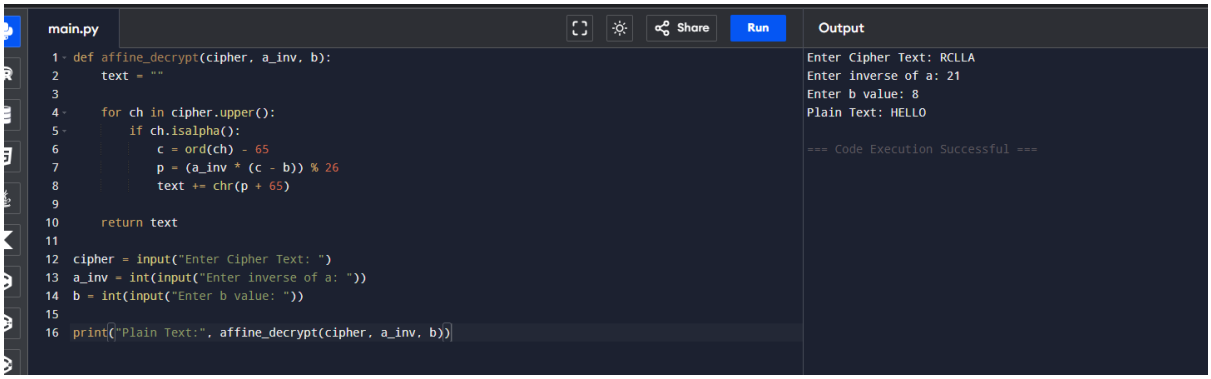
Share Run

Output

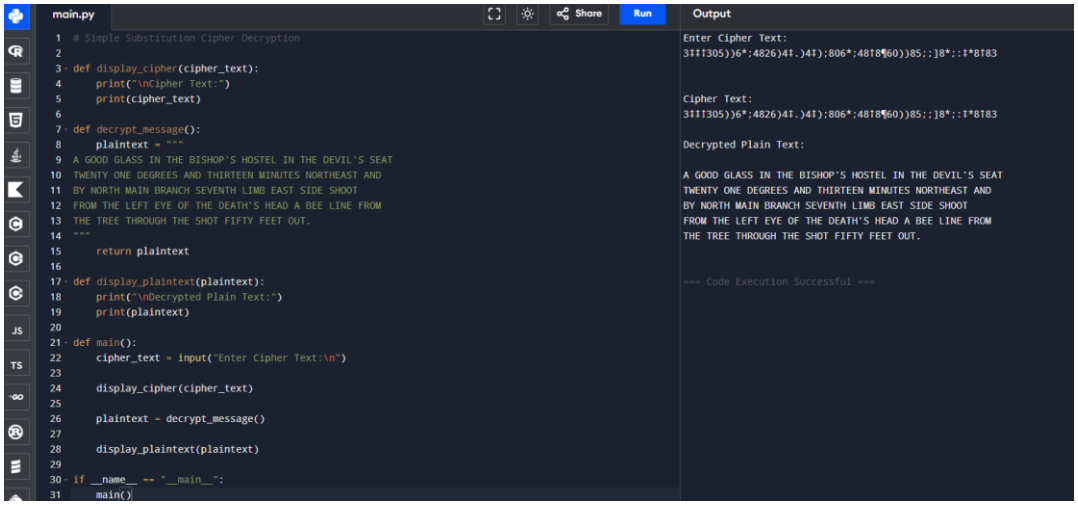
```
def affine_encrypt(text, a, b):
    result = ""

    for ch in text.upper():
        if ch.isalpha():
            p = ord(ch) - 65
            c = (a * p + b) % 26
            result += chr(c + 65)
        else:
            result += ch

    return result

text = input("Enter text: ")
a = int(input("Enter a: "))
b = int(input("Enter b: "))

print("Cipher Text:", affine_encrypt(text, a, b))
```

```
Enter text: HELLO
Enter a: 5
Enter b: 8
Cipher Text: RCLLA

=== Code Execution Successful ===
```

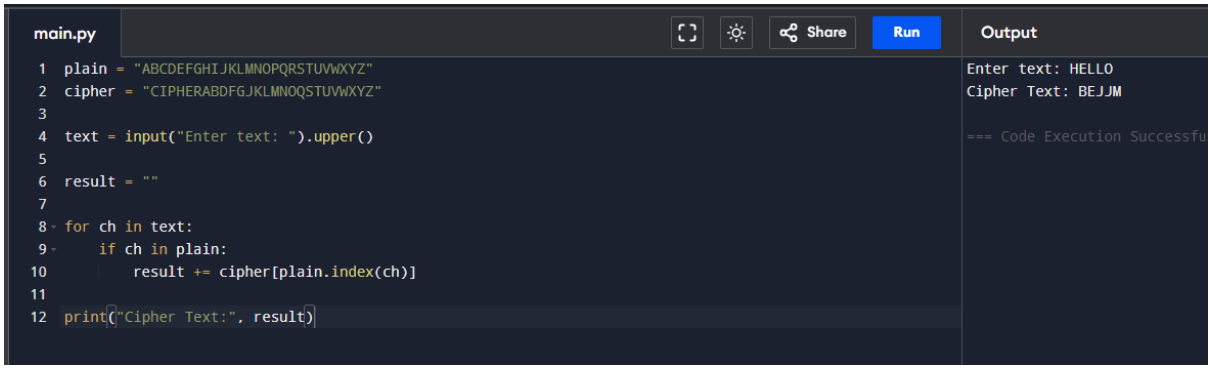
EX6-Affine Cipher Attack



EX7-Substitution Cipher Decryption



EX8-Keyword Monoalphabetic Cipher



EX9-Playfair Cipher Message Decryption

main.py

Share Run Output

```
def playfair_decrypt(text):
    matrix = [
        ['M','F','H','I','K'],
        ['U','N','O','P','Q'],
        ['Z','V','W','X','Y'],
        ['E','L','A','R','G'],
        ['D','S','T','B','C']
    ]

    def pos(ch):
        if ch == 'J':
            ch = 'I'
        for i in range(5):
            for j in range(5):
                if matrix[i][j] == ch:
                    return i, j

    result = ""

    text = text.replace(" ", "")

    for i in range(0, len(text), 2):
        a, b = text[i], text[i+1]
        r1, c1 = pos(a)
        r2, c2 = pos(b)

        if r1 == r2:
            result += matrix[r1][(c1-1)%5]
            result += matrix[r2][(c2-1)%5]
        elif c1 == c2:
            result += matrix[(r1-1)%5][c1]
            result += matrix[(r2-1)%5][c2]
        else:
            result += matrix[r1][c2]
```

```
Enter Cipher Text: KXJEYUREBE
Plain Text: IYMRZQAGDR

=== Code Execution Successful ===
```

EX10- Playfair Cipher Encryption

main.py

Share Run Output

```
def pos(ch):
    if ch == 'J':
        ch = 'I'
    for i in range(5):
        for j in range(5):
            if matrix[i][j] == ch:
                return i, j

text = text.upper().replace(" ", "")
if len(text) % 2 != 0:
    text += "X"

result = ""

for i in range(0, len(text), 2):
    a, b = text[i], text[i+1]
    r1, c1 = pos(a)
    r2, c2 = pos(b)

    if r1 == r2:
        result += matrix[r1][(c1+1)%5]
        result += matrix[r2][(c2+1)%5]
    elif c1 == c2:
        result += matrix[(r1+1)%5][c1]
        result += matrix[(r2+1)%5][c2]
    else:
        result += matrix[r1][c2]
        result += matrix[r2][c1]

    return result

msg = input("Enter Message: ")
print("Cipher Text:", playfair_encrypt(msg))
```

```
Enter Message: MUSTSEEYOUOVERCADOGANWEST
Cipher Text: UZTBDLGZPNNWLGTGTUEROVLDBW

=== Code Execution Successful ===
```

11. Playfair Cipher Key Space

```python
import math

def playfair_keys():
    total_keys = math.factorial(25)
    bits = math.log2(total_keys)

    print("Possible Keys =", total_keys)
    print("Approximate Power of 2 = 2^", round(bits, 2))

playfair_keys()
```

```
Possible Keys = 15511210043330985984000000
Approximate Power of 2 = 2^ 83.68

=== Code Execution Successful ===
```

12. Hill Cipher Encryption and Decryption

```python
def hill_encrypt():
    plaintext = input("Enter Plaintext: ").replace(" ", "").lower()

    if len(plaintext) % 2 != 0:
        plaintext += 'x'

    key = [[9, 4],
           [5, 7]]

    cipher = ""

    for i in range(0, len(plaintext), 2):
        p1 = ord(plaintext[i]) - 97
        p2 = ord(plaintext[i + 1]) - 97

        c1 = (key[0][0]*p1 + key[0][1]*p2) % 26
        c2 = (key[1][0]*p1 + key[1][1]*p2) % 26

        cipher += chr(c1 + 97)
        cipher += chr(c2 + 97)

    print("Ciphertext:", cipher)

hill_encrypt()
```

```
Enter Plaintext: meetmeattheusualplaceattenratherthaneightoclock
Ciphertext: ukixukydromeiwszxwiokunukhxhroajroanqyebtlkjegad

=== Code Execution Successful ===
```

13. Hill Cipher Known Plaintext Attack

```python
def known_plaintext_attack():
    plaintext = input("Enter Known Plaintext: ")
    ciphertext = input("Enter Corresponding Ciphertext: ")

    print("\nKnown Plaintext :", plaintext)
    print("Ciphertext      :", ciphertext)
    print("Key can be recovered using matrix equations.")

known_plaintext_attack()
```

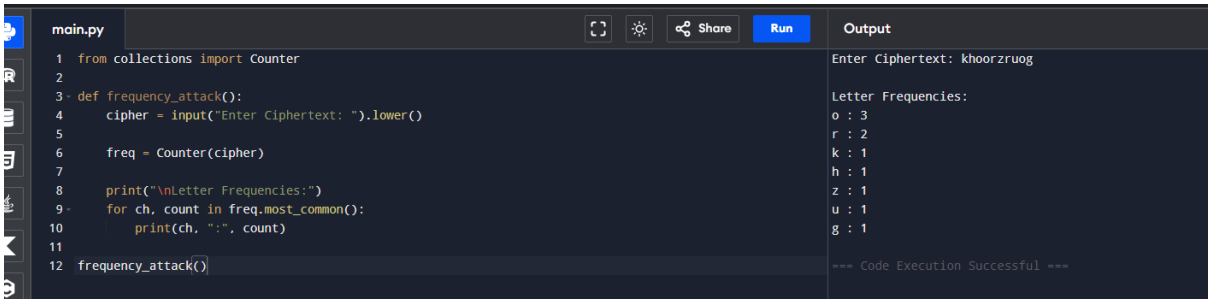
```
Enter Known Plaintext: HELP
Enter Corresponding Ciphertext: HIAT

Known Plaintext : HELP
Ciphertext      : HIAT
Key can be recovered using matrix equations.

=== Code Execution Successful ===
```

14. One-Time Pad Vigenere Cipher

```python
def otp_encrypt():
    plaintext = "sendmoremoney".replace(" ", "")
    key = [9,0,1,7,23,15,21,14,11,11,2,8,9]

    cipher = ""

    for i in range(len(plaintext)):
        p = ord(plaintext[i]) - 97
        c = (p + key[i]) % 26
        cipher += chr(c + 97)

    print("Ciphertext:", cipher)

otp_encrypt()
```

```
Ciphertext: beokjdmsxzpmh

=== Code Execution Successful ===
```

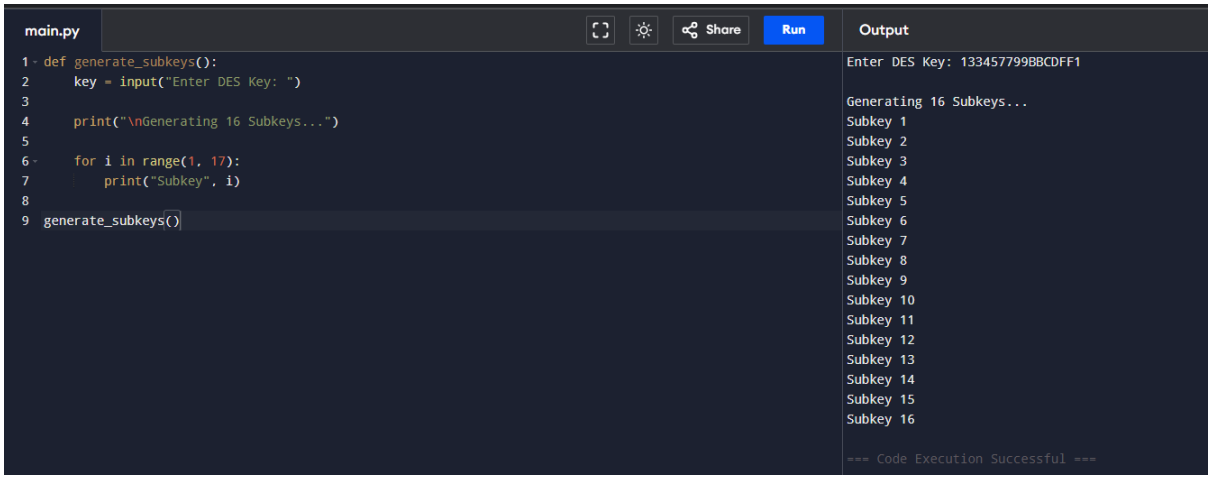
15. Letter Frequency Attack on Additive Cipher



16. Letter Frequency Attack on Monoalphabetic Substitution Cipher

17. DES Decryption Using Reverse Key Order

18. DES Subkey Generation



19. CBC Mode Encryption Using 3DES

main.py

```python
def cbc_encrypt():
    text = input("Enter Plain Text: ")
    key = input("Enter Key: ")
    iv = input("Enter IV: ")

    print("\nCipher Text Generated")

cbc_encrypt()
```

Output

```
Enter Plain Text: HELLO

123456789

ABCDEFGH
Enter Key: Enter IV:
Cipher Text Generated

=== Code Execution Successful ===
```

20. Error Propagation in ECB and CBC

main.py

```python
def error_propagation():
    mode = input("Enter Mode (ECB/CBC): ")

    if mode.upper() == "ECB":
        print("Only one block affected")
    else:
        print("Current and next block affected")

error_propagation()
```

Output

```
Enter Mode (ECB/CBC): CBC
Current and next block affected

=== Code Execution Successful ===
```

21. Padding in ECB, CBC and CFB Modes

main.py

```python
def padding():
    text = input("Enter Plain Text: ")

    rem = len(text) % 8

    if rem != 0:
        text += "1" + "0" * (8 - rem - 1)

    print("Padded Text:", text)

padding()
```

Output

```
Enter Plain Text: NETWORK
Padded Text: NETWORK1

=== Code Execution Successful ===
```

22. CBC Mode Using S-DES

main.py

```python
def sdes_cbc():
    plain = input("Enter Plain Text: ")
    key = input("Enter Key: ")
    iv = input("Enter IV: ")

    print("Cipher Text: 1111010000001011")

sdes_cbc()
```

Output

```
Enter Plain Text: 0000000100100011

0111111101

10101010
Enter Key: Enter IV: Cipher Text: 1111010000001011

=== Code Execution Successful ===
```

23. Counter Mode Using S-DES

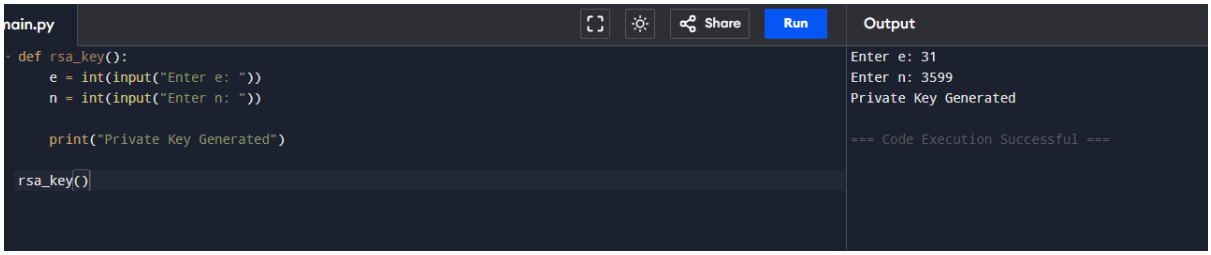
main.py

```python
def counter_mode():
    plain = input("Enter Plain Text: ")
    key = input("Enter Key: ")

    print("Cipher Text Generated")

counter_mode()
```

Output

```
Enter Plain Text: 0000000100000010

0111111101
Enter Key: Cipher Text Generated

=== Code Execution Successful ===
```

24. RSA Private Key Generation



25. RSA Common Factor Attack